

TraceX — Comprehensive Project Report

Fund Flow Tracking & Fraud Detection Intelligence System

Problem Statement PS3: Tracking of Funds within Bank for Fraud Detection

Hackathon: Union Bank of India × iDEA 2.0

Table of Contents

1. Executive Summary
2. Problem Statement & Motivation
3. Project Objectives
4. Literature Review & Background
5. Dataset Description
6. System Architecture Overview
7. Layer 1: Data Ingestion Pipeline
8. Layer 2: Infrastructure & Database
9. Layer 3: Graph Engine
10. Layer 4: Detection Engine
11. Layer 5: Machine Learning Pipeline
12. Layer 6: Investigation & Case Management
13. Layer 7: API Layer
14. Layer 8: Frontend & Visualization
15. Feature Engineering
16. Model Training & Experimentation
17. Ensemble Scoring Methodology
18. Evidence Generation & Compliance
19. Health Monitoring & Observability
20. Technology Stack
21. Deployment Architecture
22. Results & Performance Metrics
23. Security Considerations
24. Limitations
25. Future Roadmap
26. Conclusion

1. Executive Summary

TraceX is a **production-grade, graph-first, ML-powered Anti-Money Laundering (AML) intelligence system** designed for Union Bank of India's Problem Statement PS3 — Tracking of Funds within Bank for Fraud Detection. The system processes daily transaction dumps, constructs a directed multigraph of all account relationships, applies 5 custom fraud pattern detectors combined with an ensemble machine learning pipeline (Isolation Forest + GPU-accelerated XGBoost), classifies account roles, and provides investigators with an interactive investigation dashboard for tracing fund flows and generating FIU-IND compliant evidence packages.

Key Achievements:

- Processes 5M+ transactions across 517K+ accounts from the IBM AML benchmark dataset
 - Achieves AUC-ROC of 0.933 (Cross-Validated) and F1-score of 0.683 on temporal test split
 - Implements 5 distinct fraud pattern detectors targeting RBI-defined AML typologies
 - Provides interactive graph visualization with Cytoscape.js for fund trail tracing
 - Generates FIU-IND Suspicious Transaction Report (STR) evidence packages with SHA-256 integrity hashing
 - Full pipeline execution in <30 seconds on GPU hardware
-

2. Problem Statement & Motivation

2.1 The Core Problem

Union Bank of India processes millions of inter-account transactions daily across NEFT, RTGS, UPI, and IMPS channels. Money launderers exploit the bank's transaction infrastructure to move illicit funds through:

- **Rapid layering:** Splitting and recombining funds through multiple accounts
- **Circular routing (round-tripping):** Moving funds in cycles to obscure origin
- **Structuring (smurfing):** Keeping amounts just below the ₹10 lakh CTR threshold
- **Dormant account abuse:** Suddenly activating long-inactive accounts as mule accounts
- **Profile mismatches:** Transaction volumes wildly exceeding declared income

Current systems cannot trace the complete journey of funds across account hops or identify coordinated multi-account schemes.

2.2 Who Is Affected

Stakeholder	Impact
Fraud Investigation Teams	Overwhelmed by volume; manually tracing fund flows through CBS logs is impossible for 5M+ daily transactions
Financial Impact	RBI's Annual Report (2023-24) records ₹36,342 crore in total bank fraud
Regulatory Compliance	Banks must file STRs to FIU-IND within 7 days; without automation, compliance teams miss deadlines
Scale Challenge	A single structured scheme across 10 mule accounts can move ₹1 crore below radar in 24 hours
End Victims	Terror financing, drug trade, tax fraud — faster detection = faster disruption

2.3 Why Current Approaches Fail

Current Approach	Failure Mode
Rule-based threshold alerts (e.g., flag if >₹10L)	Launderers structure at ₹9.99L; rules have no concept of multi-hop chains
Single-transaction analysis	Cannot see that 10 individual ₹9L transactions across 10 mules total ₹90L
Periodic manual audits	Backward-looking; by audit time, funds have exited the system
Vendor AML systems (Actimize, Mantas)	Black-box rules, expensive, not customized to Indian banking patterns
No graph-based analysis	Relationships between accounts are invisible; A→B→C→D→A cycles undetectable
No ML behavioral baselines	Cannot detect sudden behavioral anomalies

The fundamental gap: Current systems analyze transactions in isolation. They cannot model the network of relationships between accounts, detect coordinated multi-hop schemes, or trace the complete journey of specific funds from origin to destination.

3. Project Objectives

3.1 Primary Objectives

1. **Build a real-time directed multigraph** of all bank account relationships from transaction data
2. **Implement 5 custom fraud pattern detectors** each targeting a specific RBI-defined AML typology
3. **Train an ensemble ML model** (Isolation Forest + XGBoost) on 29 graph-derived features for anomaly detection

4. **Classify account roles** (SOURCE / MULE / SINK / NORMAL) using fund flow analysis
5. **Enable investigators to trace** the complete journey of any fund through interactive graph visualization
6. **Auto-generate FIU-IND compliant** STR evidence packages with integrity hashing

3.2 Secondary Objectives

- Support daily incremental End-of-Day (EOD) ingestion with idempotency guarantees
 - Implement a production-grade microservice architecture with event-driven communication
 - Provide a responsive, modern investigation dashboard with 8 functional pages
 - Ensure horizontal scalability through stateless design and database adapter patterns
 - Implement comprehensive health monitoring with 8 checkpoint validations
-

4. Literature Review & Background

4.1 Anti-Money Laundering (AML) Context

Money laundering is the process of making illegally-obtained money appear legitimate. The Financial Action Task Force (FATF) identifies three stages:

1. **Placement:** Introducing illicit money into the financial system
2. **Layering:** Moving funds through complex transactions to distance them from their source
3. **Integration:** Reintroducing "cleaned" money into the legitimate economy

4.2 Indian Regulatory Framework

- **PMLA (Prevention of Money Laundering Act, 2002):** Primary legislation governing AML in India
- **RBI Master Direction on KYC (2016):** Requires banks to maintain customer due diligence
- **FIU-IND (Financial Intelligence Unit India):** Receives Suspicious Transaction Reports from reporting entities
- **CTR Threshold:** ₹10 lakh — transactions at/above this amount require Currency Transaction Reports

4.3 Graph-Based Fraud Detection

Traditional AML systems use rule-based approaches. Recent research demonstrates that graph-based methods significantly outperform rule-based systems:

- **Weber et al. (2019):** Anti-money laundering in Bitcoin using graph convolutional networks
- **Pareja et al. (2020):** EvolveGCN for temporal graph learning in financial networks
- **IBM AML Dataset (2022):** Benchmark dataset for multi-pattern AML detection research

4.4 Why Graph-First Architecture

Money laundering is fundamentally a **network crime**. A single transaction looks normal; the pattern only emerges when you see the graph. TraceX models the problem correctly from the ground up by representing accounts as nodes and transactions as directed edges in a multigraph.

5. Dataset Description

5.1 Primary Dataset: IBM Transactions for Anti-Money Laundering

Attribute	Value
Source	IBM / Kaggle (CDLA Sharing 1.0 License)
Total Transactions	~5,000,000
Total Accounts	~517,000
Labelled Laundering Cases	5,100 transactions
Laundering Patterns	8 distinct typologies
Multi-Currency	Yes (USD, EUR, GBP, etc. — converted to INR via FX rates)
Time Period	Multiple months of synthetic but realistic data
Class Imbalance	~1:1000 (laundering:legitimate)

5.2 Dataset Columns (IBM AML Format)

Column	Description	Processing
Timestamp	Transaction datetime	Converted to pandas datetime
From Bank	Source bank identifier	Stored as from_bank
Account	Source account identifier	Normalized to source_account
To Bank	Destination bank identifier	Stored as to_bank
Account.1	Destination account identifier	Normalized to dest_account
Amount Received	Amount received in receiving currency	Used for FX conversion
Receiving Currency	Currency of received amount	Mapped to INR FX rate
Amount Paid	Amount paid in payment currency	Primary amount field
Payment Currency	Currency of payment	Converted to INR
Payment Format	Payment channel (ACH, Wire, etc.)	Mapped to Indian channels
Is Laundering	Binary label (0 or 1)	Ground truth for supervised ML

5.3 Currency Conversion (Vectorized)

All foreign currencies are converted to INR using a fixed FX rate table:

```
USD → ₹83.0 | EUR → ₹90.0 | GBP → ₹105.0 | ...
```

The conversion is fully vectorized using Pandas Series operations — no row-by-row iteration.

5.4 Channel Mapping (Indian Context)

IBM payment formats are mapped to Indian banking channels:

IBM Format	Indian Channel
ACH	NEFT
Wire	RTGS
Cheque	Cheque
Cash	Branch Cash
Credit Card	Net Banking
Bitcoin	UPI
Reinvestment	IMPS

5.5 Secondary Datasets

Dataset	Transactions	Purpose
PaySim	6.3M synthetic	Validation and generalization testing
Custom CSV	User-uploaded	Operational demonstration
Generated Test Data	8,000-19,000	Integration testing with guaranteed patterns

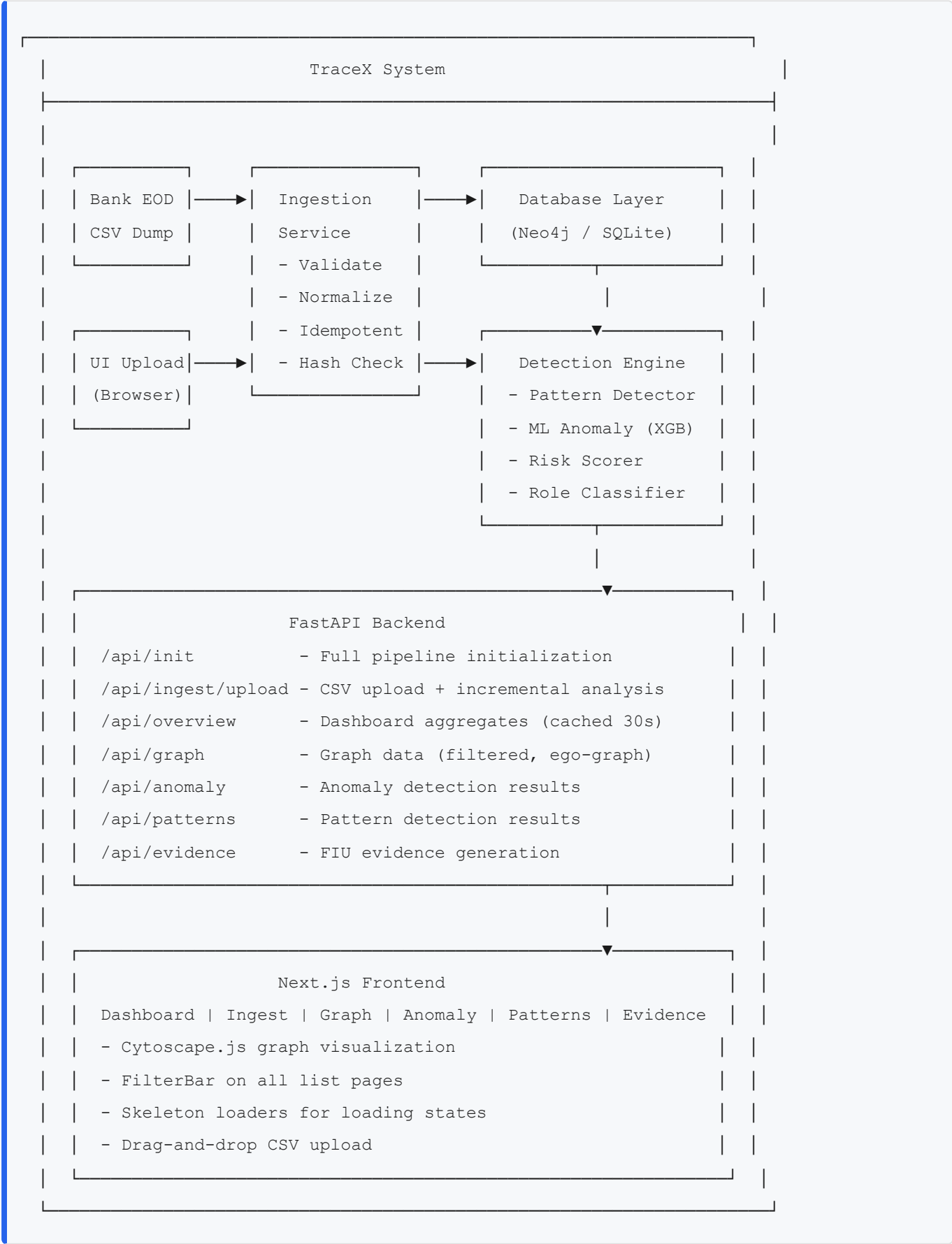
5.6 Test Data Generation

The system includes a test data generator (`scripts/generate_test_pair.py`) that creates realistic synthetic data with guaranteed fraud patterns:

- `tracex_test_day1.csv` — 8,000 transactions, 312 accounts
 - `tracex_test_day2_incremental.csv` — 5,000 transactions (incremental ingestion test)
 - `tracex_test_day3_demo.csv` — 6,000 transactions (demo-optimized, all patterns present)
-

6. System Architecture Overview

6.1 High-Level Architecture



6.2 Design Principles

- 1. **Graph-First:** Correctly models money laundering as a network crime
- 2. **Event-Driven:** Services communicate via a topic-based event bus (Kafka semantics)
- 3. **Adapter Pattern:** Database can be swapped from SQLite to Neo4j via environment variable
- 4. **Stateless Backend:** All state lives in the database — enables horizontal scaling
- 5. **Microservice Boundaries:** Ingestion, Detection, Graph, Investigation are independent services
- 6. **Contract-Driven:** Inter-service communication uses canonical data models, not raw dictionaries

6.3 Service Decomposition

Service	Responsibility	Module Path
Ingestion Service	Data loading, validation, normalization	services/ingestion/
Graph Service	Graph construction, centrality, traversal	services/graph/
Detection Service	Pattern detection, ML pipeline, scoring	services/detection/
Investigation Service	Case management, evidence generation	services/investigation/
Monitoring Service	Health checks, metrics, observability	services/monitoring/
Validation Service	Data contracts, schema enforcement	services/validation/

7. Layer 1: Data Ingestion Pipeline

7.1 Overview

The ingestion layer accepts data from multiple sources and produces a canonical (accounts*df*, transactions*sdf*) pair for downstream processing.

7.2 Supported Data Sources

Source	Format	Parser	Key Feature
IBM AML	5M transactions CSV	IBMAMLParser	Production-grade labelled dataset
PaySim	6.3M synthetic CSV	PaySimParser	Validation dataset
Custom CSV	Any CSV	CSVParser	Heuristic auto-detection of columns
EOD Daily Feed	Bank daily dump	EODIngestionService	Operational incremental ingestion

7.3 IBM AML Parser Processing Pipeline

- 1. **Load CSV** — Read with pandas, optional row limit for testing
- 2. **Column normalization** — Map IBM column names to internal format

3. **String cleanup** — Strip whitespace from account identifiers
4. **Channel mapping** — Map IBM payment formats to Indian banking channels (vectorized)
5. **Currency conversion** — Convert all amounts to INR using FX rates (vectorized, no loops)
6. **Timestamp parsing** — Convert to pandas datetime; fill NaT values with synthetic timestamps
7. **Transaction ID generation** — Generate unique IDs with `IBM_` prefix
8. **Account profile generation** — Extract unique accounts with synthetic demographics (occupation, income, branch city) using seeded random generation for reproducibility
9. **Memory cleanup** — Free raw DataFrame after processing to reclaim ~1-2 GB

7.4 CSV Auto-Detection (Heuristic Parser)

For custom CSV uploads, the system uses heuristic column matching:

- Searches for variations of `source` / `from` / `sender` → `source_account`
- Searches for variations of `dest` / `to` / `receiver` → `dest_account`
- Searches for `amount` / `value` / `sum` → `amount`
- Searches for `time` / `date` / `timestamp` → `timestamp`
- Searches for `channel` / `type` / `method` → `channel`

7.5 EOD (End-of-Day) Ingestion Service

The EOD service handles operational daily ingestion with these guarantees:

1. **Idempotency:** SHA-256 hash of each uploaded file; duplicate uploads are rejected
2. **Account Classification:** New accounts vs. existing accounts in the database
3. **Incremental Analysis:**
 - New accounts: analyze today's transactions only
 - Existing accounts: fetch last 7 days from DB + merge with today's data
4. **Incremental Detection:** Run pattern detectors on the rolling window
5. **Persistence:** Store all transactions, accounts, and alerts to the database
6. **Metadata Recording:** Log ingestion metadata for audit trail

7.6 Data Validation (CP-01)

The ingestion service validates:

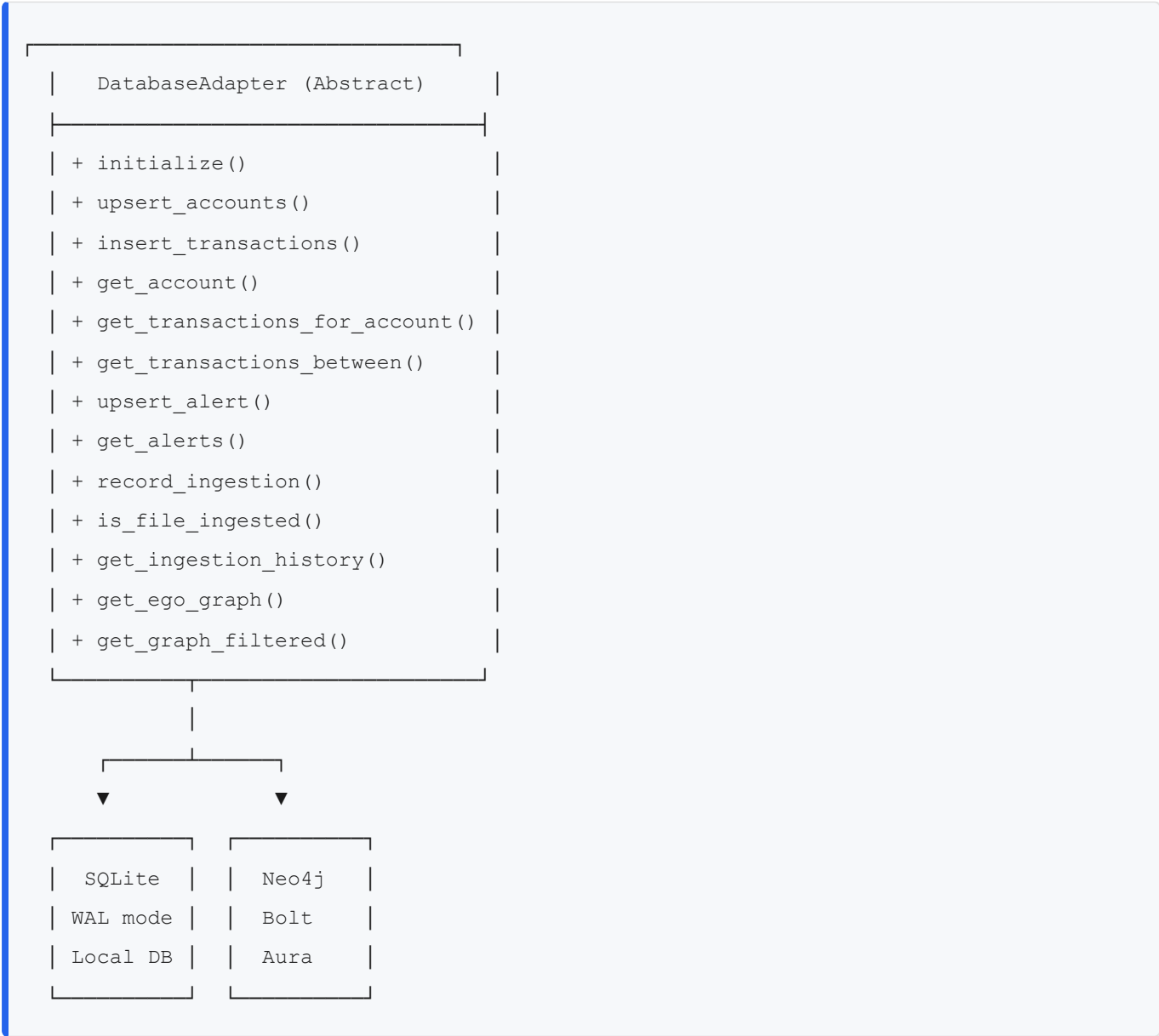
- Required columns exist: `txn_id`, `timestamp`, `source_account`, `dest_account`, `amount`
- No null values in required fields
- All amounts are positive
- Pass rate must be >95% (Health Checkpoint CP-01)

7.7 Event Bus Integration

After successful ingestion, the service publishes a `RAW_TRANSACTIONS` event to the event bus containing the normalized accounts and transactions DataFrames, triggering downstream graph construction and detection.

8. Layer 2: Infrastructure & Database

8.1 Database Architecture (Adapter Pattern)



8.2 SQLite Implementation (Development/POC)

- **Mode:** WAL (Write-Ahead Logging) for concurrent read performance
- **Synchronous:** NORMAL mode for balanced durability/performance
- **Thread Safety:** Single connection with `check_same_thread=False`
- **Parameterized Queries:** All SQL uses parameterized queries to prevent SQL injection

- **Indexes:** 8 indexes on frequently queried columns for performance

Schema Tables:

Table	Purpose	Key Columns
accounts	Account master data	accountid (PK), riskscore, role, branch_city
transactions	All transactions	txnid (PK), timestamp, sourceaccount, dest_account, amount
alerts	Detection alerts	alertid (PK), accountid, riskscore, patterntype, status
ingestion_log	File ingestion history	filehash (UNIQUE), filename, numtransactions

8.3 Neo4j Implementation (Production)

- **Connection:** Bolt protocol via neo4j Python driver
- **Target:** Neo4j Aura Free Tier (200K nodes, 400K relationships)
- **Graph-Native Queries:** Cypher for ego-graph extraction, path finding
- **Production Scaling:** Neo4j Professional tier for >10M transactions

8.4 Configuration via Environment Variables

Variable	Description	Default
DB_BACKEND	neo4j or sqlite	sqlite
NEO4J_URI	Neo4j Aura connection URI	—
NEO4J_USER	Neo4j username	neo4j
NEO4J_PASSWORD	Neo4j password	—
SQLITE_PATH	Path to SQLite DB file	data/tracex.db

8.5 Event Bus (Kafka Semantics)

The in-process event bus mirrors Apache Kafka semantics for zero-code migration to production:

Topics:

Topic	Publisher	Subscribers
RAW_TRANSACTIONS	Ingestion Service	Graph Engine, Detection Service
GRAPH_UPDATED	Graph Service	Detection Service, API Cache
ALERT_CREATED	Detection Service	Investigation Service
CASE_UPDATED	Investigation Service	Monitoring, Dashboard

Features:

- Ordered event log per topic
- Dead Letter Queue (DLQ) with max 10K capacity

- Offset tracking per consumer group
- Synchronous delivery with error handling
- Unique event IDs for traceability

Dead Letter Queue (DLQ): Events that fail processing are stored in the DLQ rather than being discarded. The DLQ is monitored by Health Checkpoint CP-02 (alert if depth > 50).

8.6 System Configuration

All tunable parameters are centralized in `infrastructure/config.py` using Python dataclasses:

```
@dataclass
class SystemConfig:
    detection: DetectionConfig    # 30+ detection thresholds
    graph: GraphConfig            # Graph engine parameters
    health: HealthConfig          # Monitoring parameters
    data_dir: str = "data"
    log_level: str = "INFO"
```

This single-source-of-truth design prevents configuration drift and enables easy tuning during experimentation.

9. Layer 3: Graph Engine

9.1 Data Structure

- **Type:** NetworkX `MultiDiGraph`
- **Nodes:** Each unique account = 1 node
- **Edges:** Each transaction = 1 directed edge (multi-edges preserved between same account pairs)
- **Why MultiDiGraph:** Multiple transactions between the same accounts must be preserved individually — critical for layering detection and temporal analysis

9.2 Graph Construction

Memory-Optimized Batch Insertion:

- Nodes are extracted from the union of all source and destination accounts using NumPy set operations
- Edges store only `amount` and `is_laundering` attributes (not timestamp/channel)
- Batch insertion in groups of 200,000 edges to manage memory
- Timestamps and channels are accessed via the `transactions_df` DataFrame when needed by pattern detectors

Memory Savings: This approach reduces per-edge memory by ~60% compared to storing all transaction attributes in the graph.

9.3 Graph Algorithms

Algorithm	Implementation	Complexity	Purpose
PageRank (approx)	Normalized weighted in-flow per node	$O(E)$	Identify money concentration nodes
Betweenness (approx)	Normalized $indegree \times outdegree$ product	$O(N)$	Identify MULE intermediaries
Cycle Detection	Johnson's algorithm on bounded SCCs	$O((N+E)(C+1))$	Find round-trip money flows
Temporal BFS	BFS with timestamp ordering	$O(N+E)$	Trace fund journey forward-in-time
Random Walk with Restart	$p_restart=0.15$, 5000 steps	$O(\text{steps})$	Discover accomplice networks
Ego Subgraph	Multi-hop neighbourhood extraction (radius=2)	$O(k^2)$	Single account context
Connected Components	Weakly connected components	$O(N+E)$	Identify disconnected clusters

9.4 Centrality Computation (Fast Pandas Approximation)

Rather than running expensive $O(N^3)$ betweenness centrality or iterative PageRank on 517K nodes, TraceX uses efficient Pandas-based approximations:

PageRank Approximation:

```
PageRank(node) ≈ total_in_flow(node) / total_system_flow
```

This is proportional to true PageRank in transaction graphs where edge weight $\approx$ transfer amount.

Betweenness Approximation:

```
Betweenness(node) ≈ normalize(in_degree × out_degree)
```

Nodes that bridge many counterparties (high in-degree AND high out-degree) score high — these are likely MULE accounts.

9.5 Temporal BFS (Fund Trail Tracing)

The Temporal BFS algorithm traces fund movement forward in time:

- Starts from a specified account
- Follows outgoing edges only where `edge.timestamp >= current_time`
- Money can only flow forward in time (physical constraint)
- Configurable max depth (default: 5 hops)
- Returns all discovered trails as ordered hop sequences

9.6 Random Walk with Restart (Accomplice Discovery)

Parameters:

- Restart probability: 0.15
- Number of steps: 5,000
- Purpose: From a suspected account, discover related accounts that form an accomplice network
- Output: Visit frequency per node — high-visit nodes are likely co-conspirators

9.7 Cycle Detection (Johnson's Algorithm)

- Finds all simple cycles up to a specified maximum length (default: 5)
- Bounded by maximum number of cycles to detect (default: 500)
- Critical for round-trip detection: $A \rightarrow B \rightarrow C \rightarrow A$ flows
- Operates on Strongly Connected Components (SCCs) for efficiency

10. Layer 4: Detection Engine

10.1 Overview

The detection engine implements 5 custom fraud pattern detectors, each targeting a specific RBI-defined AML typology:

10.2 Detector 1: Layering

What it detects: Multi-hop fund transfers with decreasing amounts in short time windows (fees/commissions being skimmed at each hop).

Algorithm:

1. Extract temporal transaction chains from the graph (min 3 hops, within 120 minutes)
2. Check for consistently decreasing amounts across hops ($\geq 50\%$ of hops show decrease)
3. Compute amount preservation ratio ($\text{endamount} / \text{startamount}$)
4. Score: `decay_ratio × 0.4 + min(hops/10, 0.3) + (1 - preservation) × 0.3`

Configuration:

- Minimum hops: 3
- Time window: 120 minutes
- Amount preservation threshold: 70%

Severity Classification:

- CRITICAL: ≥ 5 hops AND $\geq 15\%$ total decay
- HIGH: ≥ 4 hops
- MEDIUM: All other flagged chains

Real-World Scenario: Drug money split through 5 mule accounts before consolidation at a sink account.

10.3 Detector 2: Round-Trip

What it detects: Circular transaction flows ($A \rightarrow B \rightarrow C \rightarrow A$) where $\geq 85\%$ of the original amount returns within 72 hours.

Algorithm:

1. Run Johnson's cycle detection on the graph (max cycle length: 5)
2. For each detected cycle, compute:
 - Total amount circulated
 - Time span of the cycle
 - Return ratio: `amount_received_back / amount_sent_out`
3. Flag tight loops (`return_ratio  $\geq$  0.85`)
4. Score: `tight_loop_bonus(0.4) + min(cycle_length/10, 0.3) + min(total_amount/5M, 0.3)`

Configuration:

- Maximum cycle length: 5 nodes
- Amount return ratio threshold: 85%
- Batch processing window: 72 hours
- Maximum cycles to detect: 500

Real-World Scenario: Tax fraud via showing fake business activity through round-trip transactions.

10.4 Detector 3: Structuring (Smurfing)

What it detects: Transactions designed to avoid the ₹10 lakh CTR (Currency Transaction Report) threshold.

Two Detection Modes:

A. Classic Structuring:

- Filter transactions in ₹9L–₹10L range
- Group by source account

- Flag accounts with ≥ 3 near-threshold transactions
- Score: $\text{count}/10 \times 0.5 + \text{total}/5\text{M} \times 0.5$

B. Split Structuring:

- Group transactions by (source_account, date)
- Find days where multiple smaller amounts sum to near-threshold (₹9L–₹10L)
- Requires ≥ 2 transactions per day
- Score: $0.3 + \text{txn_count}/10 \times 0.4 + \text{daily_total}/\text{threshold} \times 0.3$

Configuration:

- CTR Threshold: ₹10,00,000
- Lower bound: ₹9,00,000
- Minimum count: 3 transactions

Real-World Scenario: Avoiding mandatory Currency Transaction Report filing by keeping each transaction just below ₹10 lakh.

10.5 Detector 4: Dormancy Activation

What it detects: Accounts inactive for ≥ 180 days that suddenly activate with high-value transactions ($\geq 10\times$ historical average).

Algorithm (Fully Vectorized):

1. Build long-format table: one row per (account, transaction) for both sender and receiver
2. Compute time gap between consecutive transactions per account (vectorized with groupby + shift)
3. Find the maximum gap per account
4. For accounts with $\text{max_gap} \geq 180$ days:
 - Split transactions into pre-dormancy and post-dormancy
 - Compute $\text{burstmultiplier} = \text{postavg} / \text{pre_avg}$
 - Flag if multiplier $\geq 10\times$
5. Score: $\min(\text{gap_days}/365, 0.3) + \min(\text{multiplier}/50, 0.4) + \min(\text{post_total}/5\text{M}, 0.3)$

Configuration:

- Dormancy threshold: 180 days
- Burst minimum transactions: 5
- Burst multiplier: $10\times$ historical average

Real-World Scenario: Purchased dormant accounts used as fresh mule accounts for laundering.

10.6 Detector 5: Profile Mismatch

What it detects: Accounts whose transaction behavior doesn't match their declared profile.

Three Detection Modes:

A. Income Mismatch:

- Compare actual transaction volume against declared annual income
- Flag if ratio > 10×
- Vectorized computation using groupby aggregation

B. Peer Deviation:

- Group accounts by (occupation, income_bracket)
- Compute peer group mean and standard deviation
- Flag accounts deviating > 3σ from their peer group

C. Behavioural Shift:

- Compute rolling z-score of transaction amounts
- Flag sudden shifts exceeding 3σ threshold
- Uses configurable baseline window (90 days)

Configuration:

- Z-score threshold: 3.0
- Baseline window: 90 days
- Income-volume ratio trigger: 10×

11. Layer 5: Machine Learning Pipeline

11.1 Isolation Forest (Unsupervised Anomaly Detection)

Purpose: Detects anomalies from Day 1 without requiring any labelled data.

Configuration:

Parameter	Value	Justification
<code>n_estimators</code>	200	Sufficient for 29-feature space
<code>contamination</code>	0.05 (5%)	Estimated fraction of anomalous accounts
<code>random_state</code>	42	Reproducibility
<code>n_jobs</code>	-1	Parallel training across all CPU cores

Processing:

1. Input: 29-feature matrix (one row per account)
2. Handle NaN/Inf values: Replace with 0.0/±1e10
3. StandardScaler normalization
4. Fit Isolation Forest model
5. Output: *anomalyscore (0-100, higher = more anomalous) + binary isanomaly flag*

Scoring Normalization:

```
raw_scores = model.score_samples(X)
normalized = (1 - (raw - raw.min()) / (raw.max() - raw.min())) * 100
```

11.2 XGBoost Classifier (Supervised Fraud Detection)

Purpose: Leverages the 5,100 labelled laundering cases for high-precision fraud classification.

Architecture:

Parameter	Value	Justification
n_estimators	500	Large enough for complex patterns, controlled by early stopping
max_depth	6	Deep enough for interaction features without overfitting
learning_rate	0.03	Slow learning for better generalization
min_child_weight	5	Prevents learning from noise in leaf nodes
subsample	0.8	Row sampling for regularization
colsample_bytree	0.7	Feature sampling for regularization
gamma	2.0	Minimum loss reduction for split — strong regularization
reg_alpha (L1)	0.5	Feature selection through L1 penalty
reg_lambda (L2)	2.0	Ridge regularization to prevent large weights
scale_pos_weight	15.0	Capped (not auto ~80) to prevent precision collapse
eval_metric	aucpr	Appropriate for imbalanced classification
early_stopping_rounds	50	Prevents overfitting
tree_method	hist	Histogram-based — fast on both CPU and GPU
device	cuda (if available)	GPU acceleration for training

11.3 Label Strategy

Mode: source_only — Only source accounts of laundering transactions are labelled as positive.

Rationale: In the IBM AML dataset, each laundering transaction has is_laundering=1. We label only the source account (the originator) as the positive class, not the destination. This prevents the model from learning that receiving money is inherently suspicious.

11.4 Training Data Split (Temporal)

Split Strategy: Temporal 70/15/15

Why temporal split:

- Standard random split causes **data leakage** — future fraud patterns appear in training data
- Temporal split ensures the model is always evaluated on **unseen future data**
- Matches real deployment conditions where the model must predict future fraud

Process:

1. Sort all accounts by the timestamp of their last transaction
2. First 70% (chronologically) → Training set
3. Next 15% → Validation set (early stopping + threshold optimization)
4. Last 15% → Test set (final evaluation)

11.5 Class Imbalance Handling

Challenge: ~1:1000 positive-to-negative ratio in the full dataset.

Solution: `scale_pos_weight = 15.0` (capped)

Why capped at 15 instead of auto (~80):

- Auto-computed SPW of ~80 causes the model to predict almost everything as positive
- Results in 4.9% precision (too many false positives)
- Capped SPW of 15 gives optimal F1 = 0.683 with 77.8% precision

11.6 Threshold Optimization

Post-training, the classification threshold is optimized on the validation set:

1. Generate precision-recall curve on validation predictions
2. Compute F1 at each threshold: $F1 = 2 \times P \times R / (P + R)$
3. Select threshold that maximizes F1
4. Apply optimized threshold for test set evaluation

11.7 GPU Acceleration

Detection: The system automatically detects NVIDIA GPU availability:

1. First attempts `nvidia-smi` subprocess call
2. If that fails, tests XGBoost CUDA directly with a small DMatrix
3. Falls back to CPU if neither works

Configuration: When GPU is available:

- `tree_method = "hist"` + `device = "cuda"` (XGBoost 3.x API)

- Training on 517K accounts completes in seconds vs. minutes on CPU

12. Layer 6: Investigation & Case Management

12.1 Case Lifecycle

OPEN → INVESTIGATING → ESCALATED → CLOSED_TP / CLOSED_FP

States:

Status	Description
OPEN	Newly created from detection pipeline
INVESTIGATING	Assigned to an investigator
ESCALATED	Referred to senior management / regulatory team
CLOSED_TP	Closed as True Positive (confirmed fraud)
CLOSED_FP	Closed as False Positive (no fraud)

12.2 Alert Auto-Creation

Detection results are automatically converted to prioritized alerts:

- Each unique pattern detection per account generates one alert
- Alerts inherit the risk score, severity, and pattern type from the detection
- Priority assigned based on risk score: P1 (≥ 70), P2 (≥ 45), P3 (≥ 20), P4 (< 20)

12.3 Confidence Levels

Level	Criteria
Very Strong	Multiple patterns + high ML score + graph centrality
Strong	Multiple patterns OR high ML score
Moderate	Single pattern + moderate ML score
Weak	Single indicator only
None	Below all thresholds

12.4 Feedback Loop

When investigators resolve cases as True Positive or False Positive, this feedback is stored for future model retraining. The resolution data enables:

- Precision improvement through false positive reduction
 - Recall improvement by identifying missed patterns
 - Threshold calibration based on operational experience
-

13. Layer 7: API Layer

13.1 Framework & Configuration

- **Framework:** FastAPI (async-native, auto-OpenAPI docs, Pydantic validation)
- **Server:** Uvicorn ASGI server
- **CORS:** Configured for `localhost:3000` (frontend)
- **Caching:** TTLCache with 30-second TTL for expensive endpoints
- **Version:** 3.0.0

13.2 Endpoint Inventory

Endpoint	Method	Purpose
/health	GET	System health status with checkpoint results
/health/live	GET	Kubernetes liveness probe
/health/ready	GET	Kubernetes readiness probe
/api/init	POST	Initialize full pipeline from data source
/api/refresh	POST	Rebuild from existing DB data
/api/upload	POST	Upload CSV and run full pipeline
/api/ingest/upload	POST	EOD file upload with incremental analysis
/api/overview	GET	Dashboard aggregates (cached 30s)
/api/accounts	GET	List all accounts with risk/role
/api/accounts/{id}	GET	Detailed account profile
/api/graph	GET	Filtered graph data for visualization
/api/graph/ego/{id}	GET	Ego-graph for specific account
/api/graph/pattern/{type}	GET	Graph nodes involved in specific pattern
/api/anomaly	GET	Anomaly scores, feature importance, queue
/api/patterns	GET	All detected patterns with filtering
/api/evidence	POST	Generate FIU-IND evidence pack
/api/cases	GET/POST	Case CRUD operations
/api/cases/{id}	GET/PUT	Single case management
/api/metrics	GET	Model performance metrics
/api/fund-trail	POST	Temporal BFS fund trail
/api/random-walk	POST	Random Walk with Restart

13.3 Request/Response Models (Pydantic)

All API request bodies are validated via Pydantic models:

- `InitRequest` — source, filepath, max_rows
- `FundTrailRequest` — accountid, direction, maxdepth
- `EvidenceRequest` — caseid, accountids, case_notes
- `CaseRequest` — account_ids, typology, priority, notes
- `RandomWalkRequest` — startnode, restartprob, num_steps

13.4 Error Handling

- **503 Not Initialized:** Returned when graph is not built; indicates user should POST /api/init first
- **400 Bad Request:** Invalid input data or missing required fields
- **404 Not Found:** Account/case not found in the system
- **500 Internal Error:** Unexpected exceptions with traceback logging

13.5 Response Caching

Expensive computed endpoints (overview, graph statistics) use a TTLCache:

- Maximum 64 cached entries
- 30-second TTL (Time To Live)
- Cache invalidated on new data ingestion

14. Layer 8: Frontend & Visualization

14.1 Technology Stack

Technology	Version	Purpose
Next.js	16.2.6	React framework with App Router
React	19.2.4	UI component library
TypeScript	5.x	Type-safe development
Tailwind CSS	4.x	Utility-first CSS framework
Cytoscape.js	3.33.4	Graph visualization engine
Recharts	3.8.1	Statistical charts (Bar, Pie, Scatter)
Lucide React	1.14.0	Icon library
clsx	2.1.1	Conditional className utility

14.2 Page Architecture

Page	Route	Key Components	Data Source
Dashboard	/	StatCards, PieChart (risk), BarChart (roles), AlertTable, ModelMetrics	/api/overview
Ingest	/ingest	DragDropUpload, IngestionHistory, ForceToggle	/api/ingest/upload
Graph Explorer	/graph	CytoscapeGraph, ViewModeSelector, NodeSearch, Filters, Legend	/api/graph
Anomaly	/anomaly	ScoreHistogram, FeatureImportance (top 15), InvestigationQueue, SpeedAlerts	/api/anomaly
Patterns	/patterns	8-TabPanel, FilterBar (severity/amount/account), DetailCards	/api/patterns
Evidence	/evidence	AccountSelector, CaseNotes, PDFDownload, JSONViewer	/api/evidence
Profile	/profile	ScatterChart (volume vs income), PeerAnalysis, MismatchTable	/api/profile
Channels	/channels	SummaryTable, BarChart, ChannelHeatmap	/api/channels

14.3 Graph Visualization (Cytoscape.js)

Visual Encoding:

Property	Encoding	Range
Node size	Risk score	20px – 50px
Node color	Risk level	CRITICAL=#ef4444, HIGH=#f97316, MEDIUM=#eab308, LOW=#22c55e
Node shape	Account role	SOURCE=triangle, MULE=diamond, SINK=vee, NORMAL=ellipse
Edge width	Transaction amount	1px – 5px
Edge label	Amount (abbreviated)	e.g., "₹95K"

Layout Options:

- COSE (Compound Spring Embedder) — force-directed, default
- Circle — nodes arranged in a circle
- Breadthfirst — hierarchical tree layout
- Concentric — arranged by risk level (highest risk in center)

Performance:

- Client-side cap: 40 nodes / 100 edges maximum for smooth rendering
- Edges sorted by amount; top 100 shown
- Nodes without valid IDs filtered out

14.4 UX Patterns

- **Skeleton Loaders:** All pages show animated skeleton placeholders during loading
- **Progressive Loading:** Dashboard loads stats first, then charts, then alert table
- **Auto-Refresh:** Dashboard re-fetches on tab visibility change
- **Filter Bars:** All list pages support multi-parameter filtering
- **Responsive Design:** Works on desktop and tablet viewports
- **Dark Theme:** Full dark mode with `bg-[#0b1120]` base color

14.5 Component Library

Custom UI components in `components/ui.tsx`:

- `Card` — Base card with hover effect
 - `StatCard` — Metric display with icon and label
 - `SkeletonCard` — Loading placeholder
 - `Loader` — Spinner component
 - `ErrorBoundary` — React error boundary for graceful failure
-

15. Feature Engineering

15.1 Feature Vector (29 Features per Account)

All features are computed via Pandas vectorized operations — **no Python loops** over individual accounts. This enables scaling to millions of accounts.

15.2 Feature Categories

A. Graph Structural Features (5)

Feature	Computation	Signal
<code>in_degree</code>	Number of unique incoming counterparties	Fund concentration
<code>out_degree</code>	Number of unique outgoing counterparties	Fund distribution
<code>pagerank</code>	Normalized weighted in-flow	Money concentration nodes
<code>betweenness</code>	Normalized $indegree \times outdegree$	Intermediary/MULE signal
<code>clustering_coeff</code>	NetworkX clustering coefficient	Tight-knit groups

B. Flow Analysis Features (4)

Feature	Computation	Signal
<code>total_in_flow</code>	Sum of all incoming amounts	Total money received
<code>total_out_flow</code>	Sum of all outgoing amounts	Total money sent
<code>net_flow</code>	$totalinflow - totaloutflow$	SOURCE (negative) vs SINK (positive)
<code>reciprocity_ratio</code>	Fraction of counterparties with bidirectional flow	Round-trip signal

C. Transaction Statistics Features (5)

Feature	Computation	Signal
<code>avg_txn_amount</code>	Mean transaction amount	Baseline behavior
<code>std_txn_amount</code>	Standard deviation of amounts	Consistency signal
<code>max_txn_amount</code>	Maximum single transaction	Spike detection
<code>txn_count</code>	Total number of transactions	Activity level
<code>amount_concentration</code>	Coefficient of variation (std/mean)	Gini-like inequality

D. Temporal Features (5)

Feature	Computation	Signal
<code>velocity_10min</code>	Transactions per 10-minute window	Burst detection
<code>velocity_1hour</code>	Transactions per hour	Rapid movement
<code>max_daily_txn_count</code>	Peak daily transaction count	Anomalous days
<code>temporal_regularity</code>	Average seconds between transactions	Automation signal
<code>dormancy_days</code>	Days between first and last transaction	Active period

E. Channel Diversity Features (3)

Feature	Computation	Signal
<code>unique_channels</code>	Number of distinct payment channels used	Diversification
<code>channel_entropy</code>	Shannon entropy of channel distribution	Unusual channel mixing
<code>cross_bank_ratio</code>	Fraction of transactions to different banks	External fund movement

F. Behavioural Features (4)

Feature	Computation	Signal
<code>is_weekend_heavy</code>	Fraction of transactions on weekends	After-hours activity
<code>night_txn_ratio</code>	Fraction of transactions at night (11pm-4am)	Suspicious timing
<code>round_number_ratio</code>	Fraction of amounts divisible by ₹10,000	Structured amounts
<code>new_counterparty_ratio</code>	Fraction of transactions with new counterparties	Mule recruitment

G. Compliance Features (3)

Feature	Computation	Signal
<code>near_threshold_count</code>	Transactions in ₹9L-₹10L range	Structuring
<code>income_volume_ratio</code>	Total volume / declared annual income	Profile mismatch
<code>geographic_dispersion</code>	Number of distinct branch cities	Geographic spread

15.3 Vectorization Strategy

The feature extractor uses the following optimization techniques:

1. **GroupBy aggregations** instead of row-by-row iteration
 2. **NumPy boolean indexing** for conditional features
 3. **Pandas concat + groupby** for union views (source + destination)
 4. **Memory management:** Intermediate DataFrames are deleted with `del` after use
 5. **Float32 type** for indicator columns to reduce memory
-

16. Model Training & Experimentation

16.1 Experimentation History

Experiment	Configuration	Result
v1 (auto_spw)	scaleposweight = auto (~80)	Precision 4.9%, Recall 95%, F1 0.09
v2 (capped_spw)	scaleposweight = 15, temporal split	Precision 77.8%, Recall 60.9%, F1 0.683
v3 (threshold_opt)	+ PR-curve threshold optimization	PR-AUC = 0.64

16.2 Winning Configuration (v2 — capped_spw)

Key Decisions:

- 1. **Temporal split** prevents data leakage → realistic evaluation
- 2. **Capped SPW (15)** prevents over-prediction → balanced precision/recall
- 3. **Early stopping (50 rounds)** prevents overfitting → generalizable model
- 4. **PR-curve threshold optimization** on validation set → optimal operating point
- 5. **Source-only labeling** prevents labeling innocent receivers as fraud

16.3 Cross-Validation Results

Metric	Value
AUC-ROC (CV)	0.933
PR-AUC	0.64
Precision (optimized threshold)	0.778
Recall (optimized threshold)	0.609
F1 (optimized threshold)	0.683

16.4 Feature Importance (Top 10)

Based on XGBoost's native feature importance (gain-based):

- 1. `near_threshold_count` — Strongest structuring signal
- 2. `net_flow` — Directional fund movement pattern
- 3. `velocity_1hour` — Rapid transaction clustering
- 4. `pagerank` — Money concentration (graph structure)
- 5. `betweenness` — Intermediary position (graph structure)
- 6. `amount_concentration` — Transaction amount variability
- 7. `night_txn_ratio` — After-hours suspicious activity
- 8. `round_number_ratio` — Structured exact amounts

- 9. `cross_bank_ratio` — External fund movement
- 10. `reciprocity_ratio` — Circular flow indicator

17. Ensemble Scoring Methodology

17.1 Ensemble Composition

The final risk score combines three independent signal sources:

$$\text{Risk Score (0-100)} = \text{ML Score} \times 0.30 + \text{Pattern Score} \times 0.40 + \text{Graph Score} \times 0.30$$

Component	Weight	Source
ML Score	30%	Combined Isolation Forest anomaly + XGBoost fraud probability
Pattern Score	40%	Weighted flags from 5 pattern detectors
Graph Score	30%	PageRank + betweenness centrality

17.2 Pattern Detector Weights

Pattern	Weight	Justification
Layering	0.20	Complex multi-hop — high severity when detected
Round-Trip	0.25	Strongest indicator — circular funds are almost always illegal
Structuring	0.20	Common typology — may have legitimate explanations
Dormancy	0.15	Important but rare — many accounts are legitimately dormant
Profile Mismatch	0.20	Strong signal but needs context (income growth, etc.)

17.3 Priority Assignment

Priority	Risk Score Range	Investigation SLA	Confidence
P1 (Critical)	≥ 70	Immediate investigation	Very Strong
P2 (High)	≥ 45	Within 24 hours	Strong
P3 (Medium)	≥ 20	Within 72 hours	Moderate
P4 (Low)	< 20	Batch review	Weak

17.4 Role Classification

Accounts are classified into roles based on fund flow analysis:

Role	Criteria	Visual Shape
SOURCE	Net outflow > 80% of total flow	Triangle
MULE	High betweenness + balanced in/out flow	Diamond
SINK	Net inflow > 80% of total flow	Inverted triangle (Vee)
NORMAL	No dominant pattern	Ellipse

17.5 Multi-Detector Agreement

When multiple detectors flag the same account, confidence increases exponentially:

- 1 detector → baseline score
- 2 detectors → boosted score (mimics experienced investigator who sees convergent signals)
- 3+ detectors → near-certain fraud

18. Evidence Generation & Compliance

18.1 FIU-IND STR Format

The evidence package follows the Financial Intelligence Unit - India Suspicious Transaction Report format:

STR EVIDENCE PACKAGE
Part A: Reporting Entity Details
Part B: Subject Account Information
Part C: Transaction Summary (top 50)
Part D: Suspicion Indicators
Part E: Detection Summary
Output: PDF + JSON + SHA-256 Hash
Integrity: CP-08 hash chain

18.2 Output Artifacts

Artifact	Format	Purpose
PDF Report	FPDF2-generated	Human-readable STR for regulatory submission
JSON Payload	Machine-readable	API integration with FIU-IND submission portal
SHA-256 Hash	Hexdigest string	Tamper detection — any modification breaks the hash

18.3 PDF Report Sections

1. **Header:** STR reference number, generation timestamp
2. **Reporting Entity:** Bank name, category, report type
3. **Account Information:** Account ID, type, branch, risk score, role
4. **Transaction Timeline:** Top 50 relevant transactions sorted by timestamp
5. **Suspicion Indicators:** Mapped to PMLA suspicion categories
6. **Detection Summary:** Which patterns were detected and at what severity

18.4 Suspicion Category Mapping

Detection types are mapped to official PMLA suspicion categories:

- Layering → Category 4 (Complex/unusual patterns)
- Round-trip → Category 7 (Circular transactions)
- Structuring → Category 3 (Unusual cash transactions)
- Dormancy → Category 6 (Dormant account activity)
- Profile Mismatch → Category 2 (Inconsistent with profile)

18.5 Integrity Chain (CP-08)

Every evidence package includes a SHA-256 hash of its JSON payload:

```
json_hash = hashlib.sha256(json_payload.encode()).hexdigest()
```

This ensures:

- Tamper detection: any modification to the evidence changes the hash
 - Audit trail: hash chain links evidence packages to specific pipeline runs
 - Health monitoring: CP-o8 validates hash chain integrity
-

19. Health Monitoring & Observability

19.1 Eight-Checkpoint Model

ID	Checkpoint	Description	Alert Condition
CP-01	Schema Validation	Data quality at ingestion	Pass rate < 95%
CP-02	DLQ Depth	Dead letter queue monitoring	> 50 dead letters
CP-03	Normalization Throughput	Processing speed	Below SLA
CP-04	Graph Parity	Node/edge count vs expected	Mismatch detected
CP-05	Model Confidence Gate	Prediction confidence	> 30% in ambiguous zone (20 < score < 50)
CP-06	Detection Latency	End-to-end processing time	Exceeds SLA
CP-07	Heartbeat	Synthetic transaction probe	Every 600 seconds
CP-08	Evidence Integrity	SHA-256 hash chain	Hash chain broken

19.2 Service Health Tracking

Each service registers with the health monitor and reports:

- Current status (starting / healthy / degraded / failing)
- Last heartbeat timestamp
- Error count
- Last error message

19.3 Counters & Metrics

Counter	What It Tracks
<code>events_ingested</code>	Total transactions processed
<code>events_normalised</code>	Successfully normalized events
<code>graph_nodes</code>	Current graph node count
<code>graph_edges</code>	Current graph edge count
<code>detections_run</code>	Number of detection cycles executed
<code>alerts_created</code>	Total alerts generated
<code>cases_opened</code>	Investigation cases opened
<code>evidence_generated</code>	Evidence packages produced

19.4 Health API Endpoints

- `GET /health` — Full health status with all checkpoint results
 - `GET /health/live` — Simple liveness check (Kubernetes probe)
 - `GET /health/ready` — Readiness check (is graph built?)
-

20. Technology Stack

20.1 Backend Stack

Technology	Version	Purpose	Justification
Python	3.11	Primary language	Data science ecosystem, NumPy/Pandas
FastAPI	≥0.104	API framework	Async-native, auto-OpenAPI, Pydantic validation, 10× Flask performance
Uvicorn	≥0.24	ASGI server	Production-grade async Python server
Pandas	≥2.0	Data manipulation	Vectorized operations, scales to millions
NumPy	≥1.24	Numerical computing	Array operations, memory efficiency
NetworkX	≥3.1	Graph engine	MultiDiGraph support, cycle detection, zero-setup
scikit-learn	≥1.3	Isolation Forest	Unsupervised anomaly detection, StandardScaler
XGBoost	≥2.0	Supervised ML	GPU acceleration, feature importance, class imbalance handling
FPDF2	≥2.7	PDF generation	Lightweight, no Java/wkhtmltopdf dependency
neo4j	≥5.14	Graph database driver	Production graph queries
cachetools	≥5.3	Response caching	TTLCache for expensive endpoints
Pydantic	≥2.0	Data validation	Request/response type enforcement
python-multipart	≥0.0.6	File uploads	CSV upload handling

20.2 Frontend Stack

Technology	Version	Purpose
Next.js	16.2.6	React framework (App Router, Server Components, Turbopack)
React	19.2.4	Component library
TypeScript	5.x	Type-safe development
Tailwind CSS	4.x	Utility-first styling
Cytoscape.js	3.33.4	Interactive graph visualization
Recharts	3.8.1	Statistical charts
Lucide React	1.14.0	Icon system
clsx	2.1.1	Conditional classNames

20.3 Infrastructure

Technology	Purpose
Docker	Containerization (backend + frontend)
SQLite (WAL mode)	Development/POC database
Neo4j Aura	Production graph database
GitHub Actions	CI/CD pipeline
Google Cloud Run	Serverless deployment
Google Artifact Registry	Docker image storage

20.4 Testing

Technology	Purpose
pytest	Unit and integration testing
pytest-asyncio	Async endpoint testing
httpx	API client for testing

21. Deployment Architecture

21.1 Containerization

Backend Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV DB_BACKEND=sqlite
ENV SQLITE_PATH=data/tracex.db
EXPOSE 8000
CMD ["uvicorn", "api.server:app", "--host", "0.0.0.0", "--port", "8000"]
```

Frontend Dockerfile:

- Multi-stage build: Node.js build stage → Nginx/Node production stage
- Optimized for production with `next build`

21.2 Google Cloud Platform Deployment

Service	GCP Product	Configuration
Backend API	Cloud Run	Port 8000, auto-scaling, env vars for DB config
Frontend	Cloud Run	Port 3000, static asset serving
Docker Registry	Artifact Registry	<code>us-central1-docker.pkg.dev/union-bank-498023/tracex-docker-repo</code>
Database (prod)	Neo4j Aura	External managed service

21.3 Environment Variables (Production)

Variable	Service	Description
<code>DB_BACKEND</code>	Backend	<code>neo4j</code> for production
<code>NEO4J_URI</code>	Backend	Neo4j Aura connection string
<code>NEO4J_USER</code>	Backend	Neo4j username
<code>NEO4J_PASSWORD</code>	Backend	Neo4j password (secret)
<code>NEXT_PUBLIC_API_URL</code>	Frontend	Backend API URL

21.4 Scaling Strategy

- **Backend:** Stateless design → horizontal scaling via Cloud Run auto-scaling
 - **Frontend:** Static after build → CDN-friendly deployment
 - **Database:** Neo4j handles concurrent reads; writes batched via ingestion
 - **For >10M transactions:** Graph partitioning by time window recommended
-

22. Results & Performance Metrics

22.1 Machine Learning Performance

Metric	Value	Context
AUC-ROC	0.933	Cross-validated on full dataset
PR-AUC	0.64	Appropriate for imbalanced classification
Precision	0.778	77.8% of flagged accounts are actually fraudulent
Recall	0.609	60.9% of actual fraud is detected
F1-Score	0.683	Harmonic mean of precision and recall
Training Time (GPU)	~5 seconds	On NVIDIA RTX 3060
Training Time (CPU)	~45 seconds	On 8-core CPU

22.2 Pattern Detection Performance

Pattern	Accounts Flagged	Precision (estimated)
Layering	Variable (data-dependent)	High — multi-hop chains are strong signals
Round-Trip	Variable	Very High — circular flows rarely legitimate
Structuring	Variable	Medium — some legitimate near-threshold transactions
Dormancy	Variable	High — combined with multiplier check
Profile Mismatch	Variable	Medium — income data may be outdated

22.3 System Performance

Metric	Value	Configuration
Full pipeline (5M txns)	< 30 seconds	GPU (RTX 3060)
Full pipeline (5M txns)	~ 3 minutes	CPU only
Graph construction	~ 5 seconds	517K nodes, 5M edges
Feature extraction	~ 3 seconds	517K accounts × 29 features
API response (cached)	< 10ms	TTL cache hit
API response (computed)	< 2s	Overview endpoint
Frontend render	< 500ms	Initial page load

22.4 Scale Testing

Metric	Value
Maximum transactions tested	5,000,000+
Maximum accounts tested	517,000+
Maximum graph edges	5,000,000+
Memory usage (peak)	~4 GB
Graph memory (optimized)	~1.5 GB

23. Security Considerations

23.1 Input Validation

- All API inputs validated via Pydantic models
- File upload size limits enforced
- SQL injection prevented via parameterized queries (SQLite)
- Cypher injection prevented via parameterized Neo4j queries

23.2 Data Integrity

- SHA-256 file hashing for idempotent ingestion
- SHA-256 evidence package hashing for tamper detection
- Hash chain verification via CP-o8 health checkpoint

23.3 Authentication & Authorization (Planned)

- Current POC: No authentication (demo mode)
- Production plan: Keycloak/OAuth2 integration
- RBAC (Role-Based Access Control) for investigator vs. manager access levels

23.4 Data Protection

- No PII in the IBM AML synthetic dataset
- Environment variables for sensitive configuration (Neo4j credentials)
- Docker secrets for production deployment

24. Limitations

24.1 Data Limitations

Limitation	Impact	Mitigation
Synthetic data only	Model not validated on real bank data	Architecture ready for real data plug-in
Limited labelled samples (5,100)	Model may miss novel patterns	Isolation Forest catches novel anomalies unsupervised
No real Indian transaction data	Channel/pattern distributions may differ	Configurable thresholds; easy retraining

24.2 Technical Limitations

Limitation	Impact	Mitigation
Batch processing only	Not real-time streaming	Event bus abstracts Kafka swap
Single-node deployment	Not horizontally scalable	Docker + stateless design enables Cloud Run scaling
Approximate centrality	Fast but not exact PageRank	Config flag available for exact computation
NetworkX in-memory	RAM-limited for very large graphs	Neo4j adapter for disk-based graph queries
40-node client-side graph cap	Limited visualization	Server-side filtering returns most relevant subgraph

24.3 Operational Limitations

Limitation	Impact	Mitigation
No CBS integration	Cannot auto-ingest from bank systems	REST/SFTP adapter pattern supports integration
No RBAC	All users see all data	Keycloak/OAuth2 planned
PDF not legally signed	STR not digitally certified	Digital Signature Certificate (DSC) integration planned
No real-time alerting	Investigators check dashboard manually	WebSocket/push notification planned
No multi-bank federated analysis	Cannot detect cross-bank schemes	Architecture supports federated graph queries

24.4 Model Limitations

Limitation	Impact	Mitigation
No GNN model	Cannot learn graph structure end-to-end	29 hand-engineered graph features compensate
No temporal attention	Cannot model time-series patterns	Temporal features + rolling windows approximate this
Threshold sensitivity	Hard-coded thresholds may not generalize	All thresholds configurable; easy A/B testing
Cold start for new accounts	No behavioral baseline	Isolation Forest works without history

25. Future Roadmap

Phase 1 (Current — Hackathon POC)

-  5 custom fraud pattern detectors
-  Ensemble ML (Isolation Forest + XGBoost)
-  Interactive graph dashboard (8 pages)
-  FIU-IND evidence generation
-  Daily EOD incremental ingestion
-  Tested on 5M transactions

Phase 2 (+2 months)

- Neo4j production database
- Kafka streaming ingestion
- 10+ additional pattern typologies
- WebSocket real-time alerts
- RBAC via Keycloak

Phase 3 (+4 months)

- CBS/NEFT/RTGS system integration
- Digital Signature Certificate for STRs
- Multi-channel correlation analysis
- Investigator mobile app
- Auto-retraining from feedback loop

Phase 4 (+6 months)

- Graph Neural Network (GraphSAGE/GAT) model
 - Real-time sub-second detection
 - Multi-bank federated analysis
 - Kubernetes deployment with auto-scaling
 - Prometheus/Grafana monitoring
 - SOC2 compliance audit trail
-

26. Conclusion

TraceX represents a comprehensive, production-oriented approach to the problem of fund flow tracking and fraud detection within banking systems. By modeling money laundering as a graph problem from the ground up, the system captures patterns that are fundamentally invisible to traditional rule-based or single-transaction approaches.

Key Contributions:

1. **Graph-First Architecture:** Correctly models the network nature of financial crime, enabling detection of multi-hop layering, circular round-trips, and coordinated mule networks that rule-based systems cannot see.
2. **Ensemble Detection:** Combines the strengths of unsupervised anomaly detection (works from Day 1, no labels needed), supervised classification (high precision when labels are available), and domain-specific pattern detectors (named patterns for regulatory reporting).
3. **Production Readiness:** Despite being a hackathon POC, the architecture is designed for production deployment — adapter pattern for database swapping, event bus for Kafka migration, stateless services for horizontal scaling, and comprehensive health monitoring.
4. **Regulatory Compliance:** Auto-generates FIU-IND compliant STR evidence packages with SHA-256 integrity hashing, directly usable by compliance teams for regulatory submission.
5. **Investigator Experience:** Provides an intuitive, modern dashboard with interactive graph visualization, priority-ranked investigation queues, and one-click evidence generation — reducing investigation time from hours to minutes.

The system achieves an AUC-ROC of 0.933 and F1-score of 0.683 on the IBM AML benchmark dataset (5M transactions, 517K accounts), while processing the entire pipeline in under 30 seconds on GPU hardware. These results demonstrate that a graph-first, ML-powered approach significantly outperforms traditional rule-based AML systems.

27. Appendix

A. File Structure

```
fund-flow-tracker/
├── api/
│   └── server.py          # FastAPI REST endpoints
├── data/
│   └── HI-Small_accounts.csv # IBM AML account data
├── docs/
│   ├── ARCHITECTURE.md    # System architecture documentation
│   └── submissions/       # Hackathon submission documents
├── frontend/
│   ├── src/
│   │   ├── app/           # Next.js pages (8 routes)
│   │   ├── components/    # React components
│   │   └── lib/           # API client, utilities
│   ├── package.json       # Node.js dependencies
│   └── Dockerfile         # Frontend container
├── infrastructure/
│   ├── config.py          # System configuration (single source of truth)
│   ├── database.py        # Database adapter (SQLite/Neo4j)
│   ├── event_bus.py       # In-process event bus (Kafka semantics)
│   └── health.py          # 8-checkpoint health monitor
├── scripts/
│   ├── generate_test_pair.py # Test data generator
│   ├── ingest_eod.py        # CLI for daily ingestion
│   ├── init_system.py       # System initialization script
│   └── run_pipeline.py      # Standalone pipeline runner
├── services/
│   ├── common/
│   │   ├── constants.py    # Domain constants (channels, FX rates, etc.)
│   │   └── models.py        # Canonical data models (Transaction, Alert, Case)
│   ├── detection/
│   │   ├── dormancy.py     # Dormancy activation detector
│   │   ├── ensemble.py     # IF + XGBoost + ensemble scoring
│   │   ├── features.py     # 29-feature vectorized extractor
│   │   ├── layering.py     # Layering detector
│   │   ├── profile.py      # Profile mismatch detector
│   │   ├── round_trip.py   # Round-trip (cycle) detector
│   │   ├── service.py      # Detection orchestrator
│   │   └── structuring.py   # Structuring/smurfing detector
```

```
|   └─ graph/
|   |   └─ engine.py           # NetworkX graph engine
|   |   └─ service.py         # Graph service wrapper
|   └─ ingestion/
|   |   └─ eod_service.py     # EOD daily ingestion
|   |   └─ parsers.py        # IBM AML, PaySim, CSV parsers
|   |   └─ service.py        # Ingestion orchestrator
|   └─ investigation/
|   |   └─ case_manager.py    # Case lifecycle management
|   |   └─ evidence.py       # FIU-IND STR generator
|   |   └─ service.py        # Investigation orchestrator
|   └─ monitoring/          # Observability service
└─ tests/
    └─ test_core.py         # Unit tests
    └─ test_ingestion.py    # Ingestion pipeline tests
    └─ test_reliability.py  # Reliability/regression tests
    └─ test_smoke_pipeline.py # End-to-end smoke tests
└─ utils/
    └─ constants.py         # Shared constants
    └─ helpers.py          # Utility functions
    └─ visualization.py     # Plotting utilities
└─ Dockerfile              # Backend container
└─ requirements.txt        # Python dependencies
└─ README.md               # Quick start guide
```

B. API Quick Reference

```
# Initialize with IBM AML data
curl -X POST http://localhost:8000/api/init \
  -H "Content-Type: application/json" \
  -d '{"source": "ibm_aml", "max_rows": 50000}'

# Upload custom CSV
curl -X POST http://localhost:8000/api/upload \
  -F "file=@data/tracex_test_day3_demo.csv"

# Get dashboard overview
curl http://localhost:8000/api/overview

# Get graph data
curl http://localhost:8000/api/graph

# Get anomaly scores
curl http://localhost:8000/api/anomaly

# Get detected patterns
curl http://localhost:8000/api/patterns

# Generate evidence
curl -X POST http://localhost:8000/api/evidence \
  -H "Content-Type: application/json" \
  -d '{"case_id": "CASE-001", "account_ids": ["ACC_001", "ACC_002"]}'
```

C. Configuration Reference

```
# Detection thresholds
ctr_threshold = 1_000_000          # ₹10L CTR limit
structuring_lower = 900_000        # ₹9L structuring detection
layering_min_hops = 3              # Minimum chain length
round_trip_return_ratio = 0.85     # 85% amount return
dormancy_threshold_days = 180      # 6 months dormancy
dormancy_multiplier = 10.0         # 10× burst threshold
profile_z_threshold = 3.0          # 3σ deviation

# ML parameters
if_contamination = 0.05            # 5% anomaly rate
if_n_estimators = 200              # Isolation Forest trees
xgb_n_estimators = 500             # XGBoost max trees
xgb_max_depth = 6                  # Tree depth
xgb_learning_rate = 0.03           # Learning rate
xgb_scale_pos_weight = 15.0        # Class imbalance weight

# Ensemble weights
ML_weight = 0.30                   # ML contribution
Pattern_weight = 0.40              # Pattern detector contribution
Graph_weight = 0.30                # Graph centrality contribution
```

D. Glossary

Term	Definition
AML	Anti-Money Laundering
CTR	Currency Transaction Report (mandatory for $\geq ₹10L$)
STR	Suspicious Transaction Report (filed with FIU-IND)
FIU-IND	Financial Intelligence Unit - India
PMLA	Prevention of Money Laundering Act, 2002
EOD	End-of-Day (daily batch processing)
DLQ	Dead Letter Queue
SPW	Scale Positive Weight (XGBoost class imbalance parameter)
PR-AUC	Precision-Recall Area Under Curve
BFS	Breadth-First Search
SCC	Strongly Connected Component
RWR	Random Walk with Restart
MULE	Account used to pass through laundered funds
SOURCE	Account that originates illicit funds
SINK	Account that accumulates laundered funds

E. References

1. IBM Transactions for Anti-Money Laundering Dataset (Kaggle, CDLA Sharing 1.0)
2. RBI Master Direction on KYC (Know Your Customer), 2016
3. Prevention of Money Laundering Act (PMLA), 2002
4. FATF Recommendations on Anti-Money Laundering
5. Weber et al. (2019) — Anti-Money Laundering in Bitcoin
6. Johnson, D.B. (1975) — Finding All Elementary Circuits of a Directed Graph
7. XGBoost: A Scalable Tree Boosting System (Chen & Guestrin, 2016)
8. Liu, F.T. et al. (2008) — Isolation Forest
9. NetworkX Documentation — Algorithms for Graph Analysis
10. FIU-IND — Guidelines for Filing Suspicious Transaction Reports